

Exercício Integrador — Carga de Dados, Alteração de Tabelas e Correção de Dados

Curso de Integridade Fiscal — Banco de Dados `curso_integridade_fiscal` Complementa os módulos: BULK INSERT · ALTER TABLE · UPDATE e DELETE

1. Objetivo

Este exercício integra três competências trabalhadas separadamente nos módulos anteriores:

1. **Carga de dados** de um arquivo CSV externo com `BULK INSERT`;
2. **Evolução de estrutura** de uma tabela já carregada, usando `ALTER TABLE`;
3. **Correção de dados** com base em regras de negócio fiscal, usando `UPDATE`.

O cenário simula uma rotina real de trabalho de um auditor fiscal da SEFAZ-PB: um lote de notas fiscais eletrônicas é recebido de um sistema externo, carregado numa tabela de staging, e precisa passar por um processo de triagem e correção antes de ser considerado apto para análise.

2. Revisão rápida de conceitos

Comando	Papel neste exercício
<code>BULK INSERT</code>	Importa o arquivo CSV bruto para uma tabela de staging, sem qualquer tratamento prévio dos dados
<code>ALTER TABLE</code>	Adapta a estrutura da tabela de staging para suportar o processo de triagem (novas colunas de controle)
<code>UPDATE</code>	Corrige, padroniza e sinaliza os registros com base em regras de negócio fiscal

A tabela de staging não deve ser confundida com a tabela definitiva `nfe` do banco `curso_integridade_fiscal`: ela existe apenas para receber dados brutos e passar por depuração antes de uma eventual carga final — prática comum em rotinas de ETL (Extract, Transform, Load) usadas por órgãos fiscais para tratar arquivos recebidos de terceiros.

3. Ferramenta de trabalho: SSMS x VS Code (extensão MSSQL)

Antes de iniciar, vale registrar uma mudança importante no ecossistema de ferramentas SQL Server: o **Azure Data Studio foi oficialmente aposentado em 28/02/2026** e não recebe mais atualizações. A Microsoft consolidou o desenvolvimento SQL em duas frentes:

Critério	SQL Server Management Studio (SSMS)	Visual Studio Code + extensão MSSQL
Plataforma	Somente Windows	Windows, macOS, Linux

Critério	SQL Server Management Studio (SSMS)	Visual Studio Code + extensão MSSQL
Foco principal	Administração completa (SQL Server Agent, segurança, backups, jobs)	Desenvolvimento de consultas e scripts do dia a dia
Interface	Tradicional, orientada a objetos do banco (Object Explorer)	Editor de código moderno, leve, com IntelliSense
Extensibilidade	Limitada	Alta (mesmo ecossistema de extensões do VS Code)
Controle de versão (Git)	Básico	Integrado nativamente
Indicação de uso	Tarefas administrativas, jobs agendados, gestão de permissões	Escrita e teste de scripts, exercícios como este, trabalho colaborativo com Git

Recomendação prática para este curso: use o **SSMS** quando o exercício envolver tarefas de administração (como configurar permissões ou agendar rotinas), e o **VS Code com a extensão MSSQL** para o trabalho cotidiano de escrever, testar e versionar os scripts deste exercício. Ambos se conectam à mesma instância do SQL Server e executam os mesmos comandos T-SQL — a escolha é uma questão de fluxo de trabalho, não de compatibilidade.

Para instalar a extensão no VS Code: acesse a aba **Extensions**, busque por "**MSSQL**" (publicada pela Microsoft) e instale. Em seguida, use o comando **MS SQL: Connect** para conectar-se à instância local ou de rede.

4. Preparando o arquivo de dados

O arquivo `nfe_importacao_lote.csv` simula um lote de 40 notas fiscais recebidas de um sistema externo, com separador `;` e as colunas:

`chave_acesso; cnpj_emitente; razao_social_emitente; cfop; ncm; valor_total; data_emissao`

O lote contém **erros propositais**, típicos de arquivos reais recebidos por órgãos fiscalizadores:

- CNPJs com menos de 14 dígitos ou zerados (`00000000000000`);
- CFOPs inválidos (fora da tabela de códigos, ou não numéricos);
- NCMs com formato incorreto (letras ou tamanho errado);
- Valores totais negativos ou iguais a zero;
- Datas de emissão em formato inválido (`32/13/2026`);
- Chaves de acesso duplicadas.

Copie o arquivo para uma pasta acessível pela instância do SQL Server (ex.: `C:\dados\nfe_importacao_lote.csv`) antes de iniciar.

5. Parte 1 — Carga com BULK INSERT (revisão)

5.1 Crie a tabela de staging `stg_nfe_importacao`:

```
CREATE TABLE stg_nfe_importacao (  
    chave_acesso          VARCHAR(44),  
    cnpj_emitente         VARCHAR(20),  
    razao_social_emitente VARCHAR(150),  
    cfop                  VARCHAR(10),  
    ncm                   VARCHAR(20),  
    valor_total           VARCHAR(20), -- carregado como texto propositalmente  
    data_emissao          VARCHAR(20)  -- carregado como texto propositalmente  
);
```

Por que VARCHAR em vez de DECIMAL ou DATE? Porque o arquivo contém valores e datas em formatos inválidos. Se as colunas fossem tipadas corretamente, o **BULK INSERT** falharia no meio da carga. Carregar como texto primeiro e validar depois é uma prática comum ao lidar com arquivos de origem não confiável — você vai corrigir os tipos na Parte 2.

5.2 Execute a carga:

```
BULK INSERT stg_nfe_importacao  
FROM 'C:\dados\nfe_importacao_lote.csv'  
WITH (  
    FIRSTROW = 2,  
    FIELDTERMINATOR = ';',  
    ROWTERMINATOR = '\n',  
    CODEPAGE = '65001'  
);
```

5.3 Confirme que as 40 linhas foram carregadas:

```
SELECT COUNT(*) AS total_linhas FROM stg_nfe_importacao;
```

6. Parte 2 — Evoluindo a estrutura com ALTER TABLE

A tabela de staging, como está, só armazena o dado bruto. Para conduzir a triagem, o auditor precisa de colunas de controle que não existiam no arquivo original.

Exercício 6.1 (conceitual). Adicione as colunas de controle abaixo à tabela `stg_nfe_importacao`, escolhendo o tipo de dado adequado para cada uma:

- `status_validacao` — indica se o registro está 'PENDENTE', 'VALIDO' ou 'INVALIDO' (valor padrão 'PENDENTE');
- `motivo_rejeicao` — texto livre explicando por que o registro foi marcado como inválido (pode ser nulo);
- `data_analise` — data em que o registro foi revisado pelo auditor (pode ser nula até a revisão ocorrer);
- `auditor_responsavel` — nome do auditor que revisou o registro (pode ser nulo).

Exercício 6.2 (intermediário). Depois de perceber que `valor_total` e `data_emissao` foram carregados como texto, adicione duas novas colunas já com o tipo correto, para receber os valores convertidos após a validação:

- `valor_total_num` `DECIMAL(15,2)`
- `data_emissao_dt` `DATE`

Note que não estamos alterando o tipo das colunas originais (`ALTER COLUMN`), e sim adicionando colunas novas para os dados convertidos. Isso preserva o dado bruto original para auditoria — uma prática recomendada ao lidar com dados de terceiros: nunca sobrescrever a fonte antes de validar.

Exercício 6.3 (desafio). Adicione uma restrição `CHECK` na coluna `status_validacao` para garantir que ela só aceite os três valores previstos (`'PENDENTE'`, `'VALIDO'`, `'INVALIDO'`).

7. Parte 3 — Corrigindo e triando os dados com UPDATE

Com a estrutura pronta, é hora de aplicar as regras de negócio fiscal para validar e corrigir os registros.

Nível conceitual

7.1 Marque como `'INVALIDO'` todos os registros cujo `cnpj_emitente` não tenha exatamente 14 caracteres, preenchendo `motivo_rejeicao` com `'CNPJ com formato invalido'`.

7.2 Marque como `'INVALIDO'` os registros cujo `valor_total` (ainda em texto) represente um valor menor ou igual a zero, com o motivo `'Valor total zerado ou negativo'`.

Nível intermediário

7.3 Para os registros com `cnpj_emitente` de 14 dígitos e `valor_total` convertível para número positivo, popule `valor_total_num` com a conversão de `valor_total` para `DECIMAL(15,2)` usando `TRY_CONVERT` (para evitar erro de conversão nos registros ainda inválidos).

7.4 Popule `data_emissao_dt` a partir de `data_emissao` usando `TRY_CONVERT(DATE, data_emissao, 23)`. Em seguida, marque como `'INVALIDO'` os registros em que essa conversão resultou em `NULL`, com o motivo `'Data de emissao invalida'`.

7.5 Marque como `'INVALIDO'` os registros cujo `cfop` não pertença à lista de CFOPs válidos para o curso: (`'5101'`, `'5102'`, `'5405'`, `'5949'`, `'6101'`, `'6102'`, `'6108'`, `'6403'`), com o motivo `'CFOP fora da tabela de codigos validos'`.

Nível desafio

7.6 Identifique registros com `chave_acesso` duplicada (mais de uma ocorrência na tabela de staging) e marque **todas as ocorrências**, exceto a primeira (pela ordem de carga), como `'INVALIDO'`, com o motivo `'Chave de acesso duplicada no lote'`. Dica: utilize uma CTE com `ROW_NUMBER()` particionada por `chave_acesso`.

7.7 Todos os registros que passaram por todas as validações acima sem serem marcados como `'INVALIDO'` devem ser atualizados para `status_validacao = 'VALIDO'`, com `data_analise = GETDATE()` e `auditor_responsavel = 'Auditoria Automatizada - Triagem Inicial'`.

7.8 (integração final). Escreva uma única consulta `SELECT` que resuma o resultado da triagem: quantidade de registros por `status_validacao`, e — para os inválidos — quantidade agrupada por `motivo_rejeicao`. Essa é a visão que um auditor apresentaria como relatório de qualidade do lote recebido.

8. Para refletir

- Por que faz sentido separar a carga bruta (staging) da tabela definitiva `nfe`, em vez de aplicar `BULK INSERT` diretamente nela?
- Em uma rotina real da SEFAZ-PB, quem deveria revisar os registros marcados como `'INVALIDO'` antes de descartá-los definitivamente?
- Que outras regras de negócio fiscal (além das aplicadas aqui) poderiam ser incorporadas a essa triagem — por exemplo, cruzamento com o cadastro de contribuintes?